MAI201 MLOps: Assignment 2
Data Validation & Testing

Weight: 10%
Type: Individual

Overview

This assignment focuses on **data validation and testing** using Great Expectations and pytest. You will work with a provided messy dataset. No model training is required.

The provided dataset, customer_data.csv, contains several data quality issues:

- Missing values in the age, email, and salary columns
- Duplicate customer records
- Out-of-range values (age greater than 120, negative salary)
- Invalid email formats and inconsistent phone number formats
- Salary stored as a string with dollar signs

Learning Outcomes

- Build an MLOps framework for continuous improvement
- Develop a data pipeline to retrain ML models (data validation component)

Tasks

Part 1: Great Expectations Setup

Install Great Expectations and initialize a Great Expectations project in your assignment folder. Configure a data source pointing to the provided CSV file. Create a new expectation suite named `customer_data_expectations`.

Part 2: Create Expectations

Create the following expectations for the dataset:

- `customer_id` – must be unique and must not be null.
- `age` – must be between 0 and 120.
- `email` – must match a valid email format using regular expressions.
- `salary` – must be present in at least 95% of rows (use the `mostly` parameter).
- `country` – must be one of the following values: USA, Canada, UK, or Australia.
- `signup_date` – must be of datetime type.
- Overall table row count – must be between 500 and 1000.

Part 3: Data Quality Report

Run the expectation suite against the messy dataset and capture the validation results. Generate and save the Great Expectations data documentation as an HTML file. Identify and document all data quality issues found, including counts per issue.

Part 4: Write pytest Unit Tests

Write pytest unit tests for three data utility functions:

- `load_csv(filepath)` – test for file not found, empty file, and successful loading.
- `clean_phone(phone)` – test various input formats and invalid inputs to ensure consistent output.
- `validate_email(email)` – test valid emails, invalid emails, and edge cases.

Part 5: Documentation

Create a report named `assignment2_report.md` containing:

- A screenshot of the Great Expectations validation results
- A list of all data quality issues found (with counts per issue)
- A screenshot of the pytest execution showing all tests passing

- A brief reflection on which data quality issue would most impact ML model performance and why

Dataset Schema

Column	Type	Description
customer_id	string	Unique identifier (must be unique and not null)
age	integer	Customer age (0 to 120)
email	string	Email address (valid format required)
salary	float	Annual salary (positive, at least 95% present)
country	string	One of: USA, Canada, UK, Australia
phone	string	Phone number (various formats to clean)
signup_date	date	Date of account creation

Submission Instructions

1. Push all code to your GitHub repository.
2. Ensure the repository is accessible.
3. Submit via the course portal: your GitHub repository URL and `assignment2_report.md`.

Grading Rubric

Criteria	Weight	Excellent	Satisfactory	Needs Improvement
Great Expectations Setup	20%	Correctly initialized and configured	Minor issues	Not working
Expectations Created	30%	All eight expectations implemented correctly	Five to seven expectations	Fewer than five expectations
Data Quality Report	20%	Issues identified, HTML generated, screenshot provided	Partial identification	Missing
pytest Unit Tests	20%	All three functions tested, all tests pass	Two functions tested	One function tested
Documentation	10%	Complete report with screenshots	Partial report	Missing